

MEDIATEK

MT8676 Hypervisor Camera User Manual

版本: 1.1
出版日期: 2024-11-22

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-10-31	Jian.Xiang	正式版
1.1	2024-11-22	Jian.Xiang	修改 1.3.1 架构章中的描述

目录

版本记录 2

目录 3

图片目录 3

1 Hypervisor Camera 4

 1.1 概述 4

 1.2 Camera Turbo 5

 1.2.1 架构 5

 1.2.2 控制流程 6

 1.3 Camera Virtualization 8

 1.3.1 架构 8

 1.4 Debug Share 8

 1.4.1 Log 8

附件一 附加条款 10

图片目录

图 1-1.架构概述 4

图 1-2. Camera MW 架构 5

图 1-3. MW 类图 6

图 1-4. MW 类图 7

图 1-5. Camera Virtualization 架构 8

1 Hypervisor Camera

1.1 概述

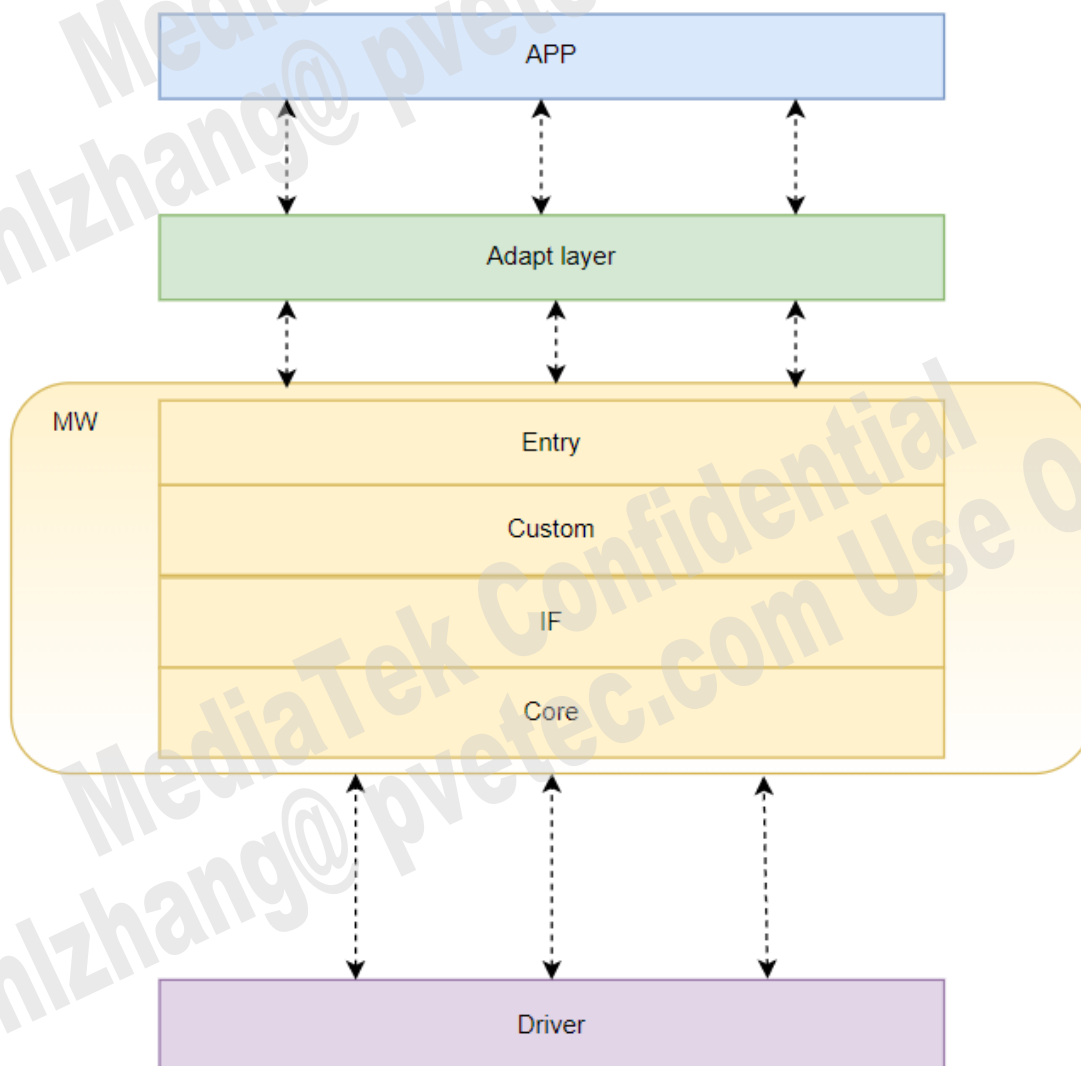


图 1-1. 架构概述

Camera Turbo 主要分为三个部分：

- 入口层（Entry）：是进入 MW 的入口。不同的操作系统可以通过调用入口层的代码通过适配层适配 MW。
- 定制层（Custom）：通过这一层，客户可以执行定制操作。
- 接口层（IF）：由入口层使用的接口层。在这里生成相机会话和原生相机的逻辑。接口层主要实现请求和配置相关操作的流程框架。
- 核心层（Core）：是 MW 的核心层。它主要与驱动程序通信，发送请求并获取结果，最后通过接口层回调到 APP。

1.2 Camera Turbo

1.2.1 架构

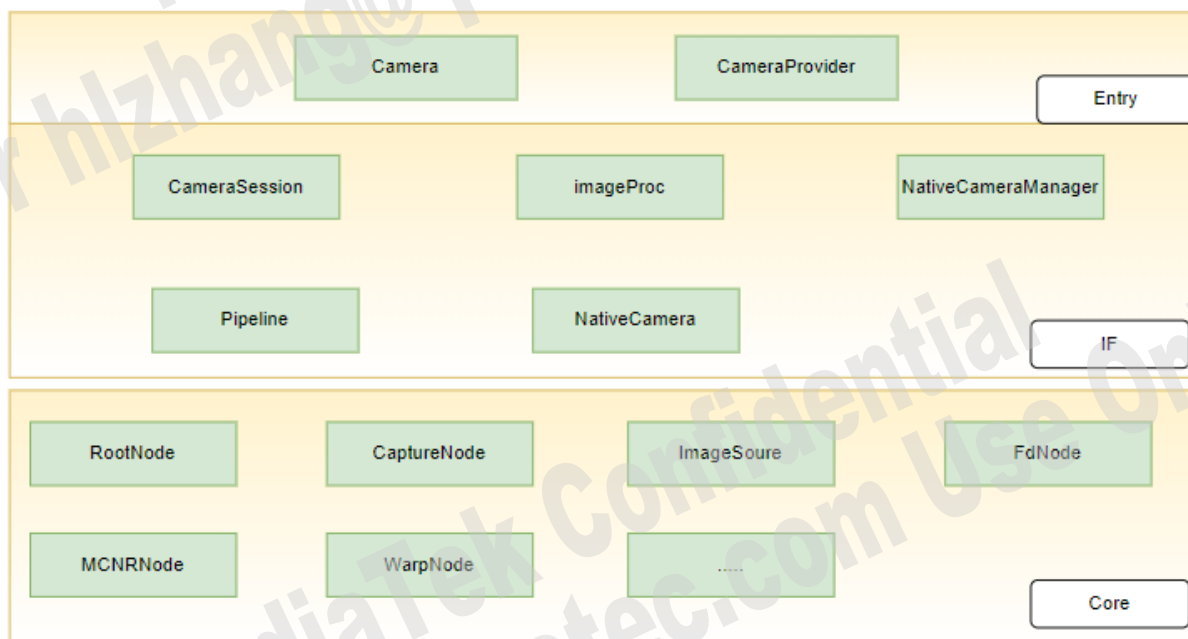


图 1-2. Camera MW 架构

Camera Turbo 的主要架构图包括入口层、接口层和核心层。定制层主要供客户使用。如果不使用，可以直接绕过。

- **入口层** 主要由 Camera 和 CameraProvider 组成。CameraProvider 主要向上层提供相机列表，并获取和控制相机的属性。Camera 主要提供接口给上层，以控制相机传感器的控制行为。
- **接口层** 主要是管线，它是构建底层节点连接关系的管理器。ImageProc 主要便于回调，为客户提供定制行为。同时，客户也可以通过 ImageProc 定制管线。
- **核心层** 主要由 ImageNode 的子类组成。这些子类与底层驱动交互，对请求实施不同的操作。例如，FdNode 主要用于面部识别，captureNode 用于相机功能，MCNRNode 用于图像处理，最终结果将通过 ImageNode 回调传递到接口层。

1.2.2 控制流程

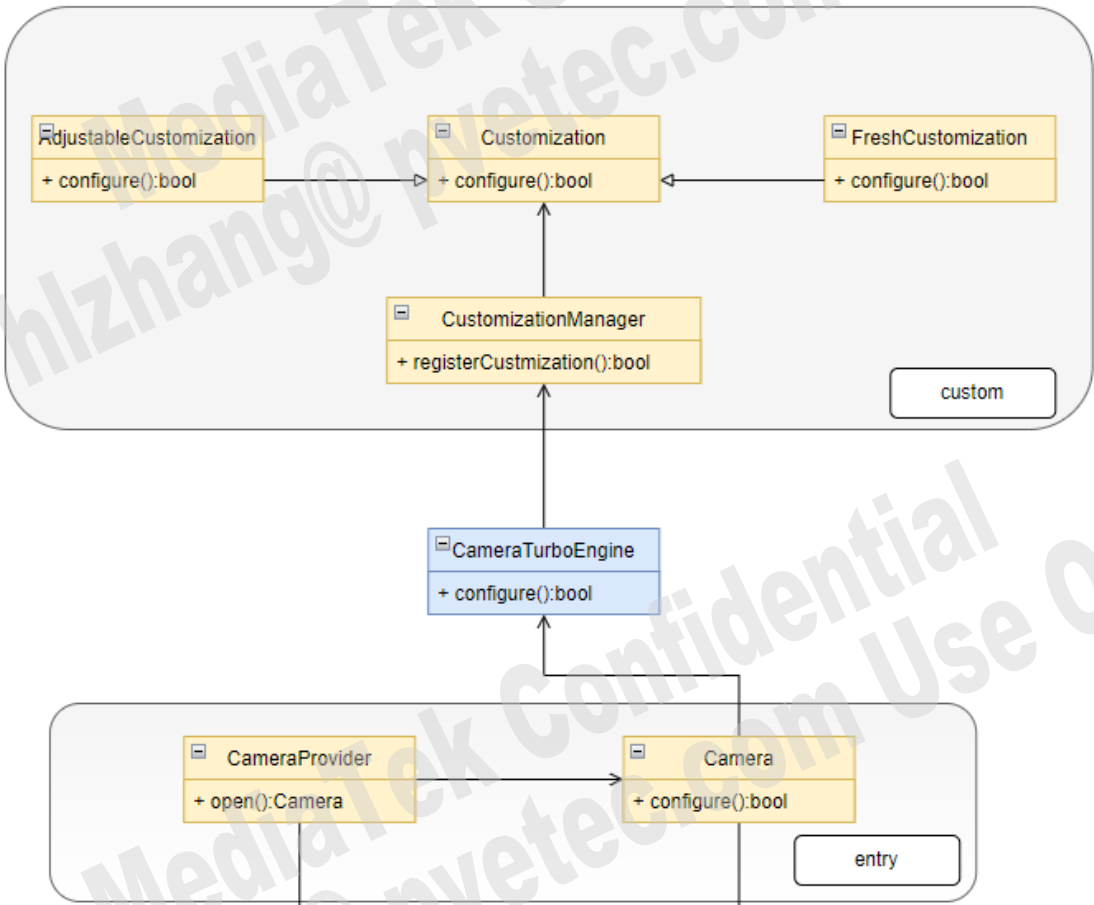


图 1-3. MW 类图

图 1-3 主要是入口层和定制层之间的类流交互图。主要是上层通过 **CameraProvider** 的 `open` 接口获取 **Camera**，**Camera** 通过调用 **Camera TurboEngine** 获取 **CustomizationManager**。**CustomizationManager** 是定制层的管理器，通过它实现对客户定制行为的管理。

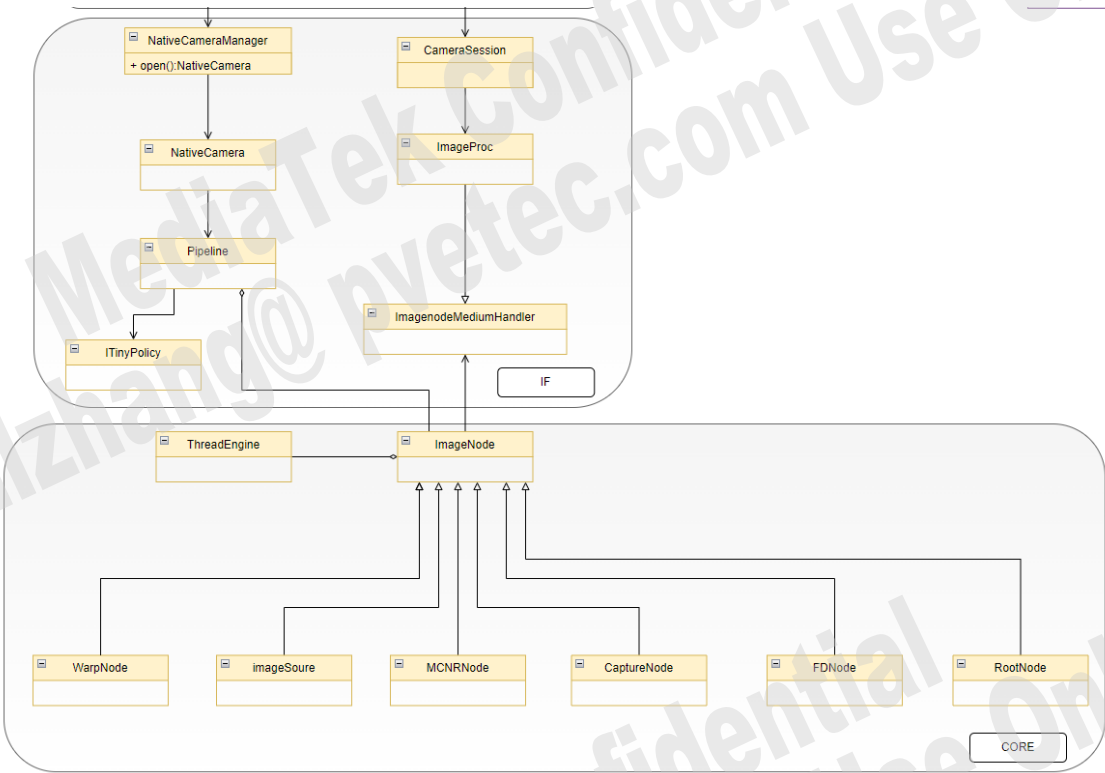


图1-4. MW 类图

接口层（IF layer）与核心层（Core）之间的交互逻辑主要是上层的 CameraProvider 的 open 接口将调用 NativeCameraManager。NativeCameraManager 分配可以在底层操作的 NativeCamera。每个 NativeCamera 将具有管线属性，这决定了每个 NativeCamera 实际运行的流程。Camera 的配置将创建一个新的 CameraSession。CameraSession 创建自己的 Pipeline。连接的 ImageProc 保存在 Pipeline 中。ImageProc 创建相应的 ImageNode。每个 CameraSession 都有自己的 ImageProc 列表。用户可以在 ImageProc 中执行定制行为。ImageProc 使用回调接口 MediumHandler。ImageNode 的回调会使用 ImageProc。

1.3 Camera Virtualization

1.3.1 架构

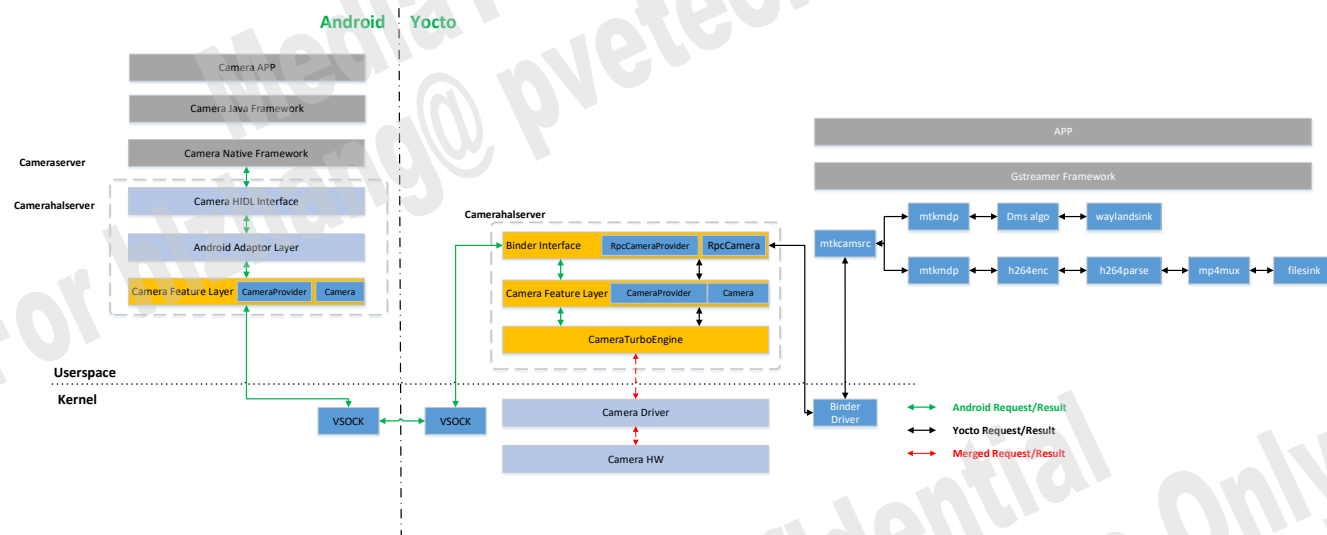


图1-5. Camera Virtualization 架构

Hypervisor 系统 guest (Android) + host (Yocto) 同时在跑，对 guest 和 host 系统的操作也是单独进行。Android 端有一个 guest camerahalservice 进程，Yocto 端有一个 host camerahalservice 进程。APP 在 Android 端，camera 底层在 Yocto 端 (mw/sensor/driver)，Android 端的 camerahalservice 以 guest 角色从 Yocto 端 camerahalservice 拿图。Yocto 端的 camerahalservice 要先跑起来，Android 的 camera APP 才能打开。当需要重启 camerahalservice 时，也是 Yocto 先重启，然后再杀掉 Android 的 camerahalservice。

1.4 Debug Share

1.4.1 Log

```
camerahalservice: [3457][TINY_NATIVECAMERAMANAGER] open cam::0 +
camerahalservice: [3457][RpcCamera] [RpcCamera] Cam::0, userId 0, logEn 0, ctor
camerahalservice: [3995][TINY_NATIVECAMERA] [operator()] Power On Sensor Success
camerahalservice: [3457][RpcCamera] [configure] Cam::0, userId 0 +
camerahalservice: [3457][TINY_CAMERA] [configure] cam[0] user[0] configure +
camerahalservice: [3457][TINY_IMAGEPROC] [addMultiCallback] [captureProc] add
callback:0x7f6002b350 for user:0
camerahalservice: [4061][TINY_NATIVECAMERA] [0][operator()] [user::0]current pipeline
active ...
camerahalservice: [4061][TINY_PIPELINE] ppl::0, active, root 0x7f60021fd0 +
camerahalservice: [3457][TinyMW/TinyPolicy] [TinyMW/TinyPolicy::initialize] +
camerahalservice: [3457][TinyMW/TinyPolicy] [TinyMW/TinyPolicy::initialize] -
camerahalservice: [3457][TINY_PIPELINE] ppl::0, init +
```

```

camerahalserver: [3457][TINY_PIPELINE] init -
camerahalserver: [3457][TINY_NATIVECAMERA] [0][configStreams] [user::0]curpipe init -
camerahalserver: [3457][TINY_NATIVECAMERA] [0][pushRequest] [user:0 R:0] +
camerahalserver: [3457][TINY_NATIVECAMERA] [0][pushRequest] [user:0 R:0] -
camerahalserver: [3994][TinyMW/TinyPolicy] [TinyMW/TinyPolicy::onEvaluateRequest]
AppRequest: 0 in Device +
camerahalserver: [3994][TINY_ROOTNODE] [0]( 86)[enqReqLoop] user:0 frameId:0
camerahalserver: [3994][TINY_IMAGENODE] [P1Node_0]( 101)[input][Cam::0] [NODEDBG_FRAME
R:0,F:0] input
camerahalserver: [3994][TINY_IMAGENODE] [ImgSrc_256]( 101)[input][Cam::0] [NODEDBG_FRAME
R:0,F:0] input
camerahalserver: [4101][TINY_IMAGEPROC] [notify] [0][P1Proc][0x7f600395b0]notify shutter
user:0 shutter:195729495000
camerahalserver: [4101][TINY_IMAGENODE] [P1Node_0]( 370)[applyRelease][Cam::0] R0:F0
camerahalserver: [3988][TINY_IMAGENODE] [McnrNode_1]( 101)[input][Cam::0] [NODEDBG_FRAME
R:0,F:0] input
camerahalserver: [4020][TINY_IMAGENODE] [McnrNode_1]( 370)[applyRelease][Cam::0] R0:F0
camerahalserver: [4020][TINY_IMAGENODE] [McnrNode_1]( 241)[applyResult][Cam::0]
[NODEDBG_FRAME R:0,F:0] out, result(0), #processing(0)
camerahalserver: [4020][CustLayer] [processCaptureResult] processCaptureResult camId:0
userId 0, frameId:0
2024-07-

```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。